PERSONALISED LEARNING
RECOMMENDATION SYSTEM

PROJECT DOCUMENTATION

Student Name	Nishant Kumar
Student Code	BWU/BCA/24/074
Department	Department of Computational Science
Programme	Bachelor of Computer Application
Subject	Machine Learning Practical
Subject Code	BCA57205
University	Brainware University

Abstract

The Personalised Learning Recommendation System is a single-file Python program that uses a decision tree to assign a learning level from academic details and then prints topic suggestions for the learner's chosen subject area. It includes a small, fixed dataset of 20 examples, a console form for new input, model evaluation output, and visual summaries of the sample data. The report explains the program in plain language, describes its structure, and records the limits that matter when interpreting its predictions.

The program is suitable as a practical demonstration of a basic machine learning workflow. Its results should be treated as educational only: the learning level is created directly from marks, and marks are also given to the model as an input. This means the model largely learns a rule already written into the program rather than discovering an independent measure of learner ability.

Report Guide

Section	What it explains
Project overview	The learning problem, aims, users, and limits.
Design and method	The sample data, model, input flow, and recommendation rules.
Implementation	The program modules, charts, setup, and example behavior.
Evaluation and next steps	What the reported metrics mean, key limitations, and improvements.

1 Introduction

Learners often need a clear next step that matches both their current understanding and the way they prefer to study. This project demonstrates one simple way to connect those ideas: it estimates a learning level, chooses a set of topics for a selected interest, and displays a resource format such as video, notes, or practical work.

This version is a command-line prototype. It asks the learner to enter marks, attendance, study hours, subject interest, and learning style. The program prints the entered profile, predicted level, and a short list of topics. It also opens charts that describe the fixed sample dataset and model evaluation.

2 Problem Statement

A learner may not know which topic to study next or which format to use. A basic recommendation tool can make that choice easier by using a learner's academic profile, subject interest, and preferred learning style. The code addresses this need with a small decision tree and predefined topic lists.

The current program is a classroom prototype rather than a complete learning platform. It does not use a live student database, verify learning progress, or fetch actual learning materials. Its predictions depend on the small examples stored in the code.

3 Objectives

- Create a small, understandable machine learning example using student marks, attendance, study hours, interest, and learning style.
- Assign each example to Beginner, Intermediate, or Advanced using a clear marks rule.
- Train and evaluate a decision tree classifier and display common classification metrics.
- Accept a learner profile from the console and predict a learning level.
- Print topic suggestions that match the selected interest and predicted level.
- Create charts that help a student inspect the sample data and model output.

4 Scope of the Project

The project covers a single user session in a terminal. It supports four interests and three learning styles. It returns four fixed topic suggestions for each interest and predicted level, followed by a format suggestion that matches the selected style.

Included	Outside the current version
Console input and range checks for numeric academic values	Graphical web or mobile interface
Decision tree training with a 75/25 holdout split	Large or institution-approved learner dataset
Predefined topic lists for 4 interests and 3 levels	Live links, resource ranking, or resource quality checks
Printed model metrics and six charts	Saved profiles, consent management, and progress history

The program is intended for a practical demonstration. It should not be used to make high-stakes academic decisions or to label real learners without better data, validation, and privacy safeguards.

5 Existing Approaches and Machine Learning Concepts

Many learning platforms suggest lessons using rules, course sequences, learner activity, or a recommender model. A rule-based system can be easy to explain but may not adjust to each learner. A data-driven model can adapt from examples, but it needs enough reliable data and careful evaluation.

The system combines a small classifier with explicit recommendation rules. The decision tree predicts one of three labels. A separate dictionary then maps the learner's chosen interest and predicted label to a fixed list of topics. Learning style changes the suggested format, not the topic list itself.

Concept	Use in this program
Classification	Predicts one label from Beginner, Intermediate, or Advanced.
Decision tree	Learns a sequence of feature splits, with maximum depth limited to 4.
Label encoding	Converts interest and style names to integer codes for the model.
Holdout evaluation	Keeps 25% of the 20 rows for a small evaluation set.
Rule-based recommendation	Selects topics from a built-in dictionary using interest and predicted level.

6 Proposed System

The proposed flow is intentionally small. The program prepares sample records and labels, encodes category values, trains a decision tree, asks the learner for a profile, predicts a level, and prints matching topics and a format suggestion. The same script also prints evaluation metrics and opens plots.

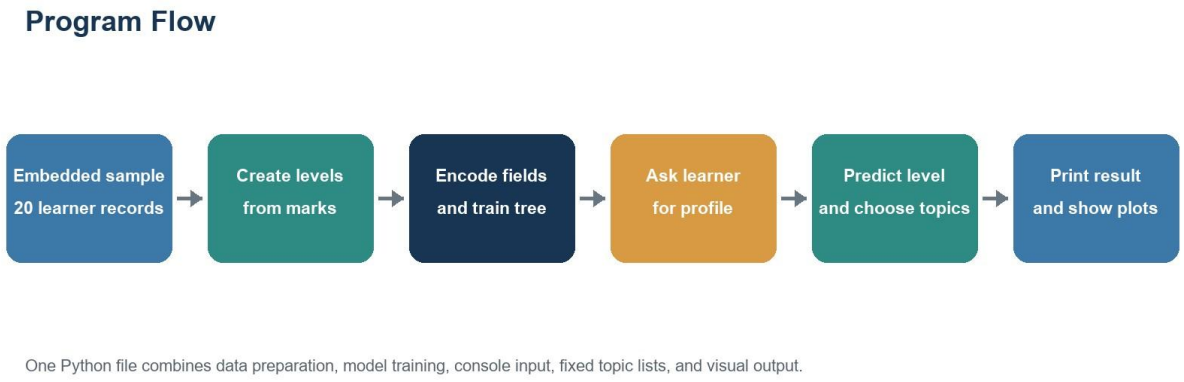


Figure 1. Main flow of the single-file Python program.

Main Features

- Numeric input for marks, attendance percentage, and daily study hours.
- Menu-based choices for Programming, Database, Web Development, or Data Science.

- Menu-based choice of Video, Notes, or Practical learning style.
- Decision tree prediction followed by topic and format suggestions.
- Evaluation printout and six visualizations generated during a run.

7 Project Methodology

Development Tools

Tool or library	Role
Python	Main programming language.
pandas	Builds and manages the 20-row data table.
scikit-learn	Splits data, encodes categories, trains the tree, and calculates metrics.
Matplotlib	Creates the plotting windows and chart layout.
Seaborn	Draws heatmaps and statistical charts with a consistent theme.
Console	Collects a profile and displays the result.

Dataset and Fields

The code creates 20 example rows directly in a Python dictionary. The program does not read a CSV file or connect to an external database. Marks and attendance range from 0 to 100 in the user interface; the stored examples cover marks 45 to 92, attendance 60 to 96, and study time 1 to 6 hours.

Field	Type	Meaning in the program
marks	Number	Academic score; also used to generate the target level.
attendance	Number	Attendance percentage.
study_hours	Number	Daily study hours.
interest	Category	Programming, Database, Web Development, or Data Science.
learning_style	Category	Video, Notes, or Practical.
Field	**Type**	**Meaning in the program**
level	Target label	Beginner below 60, Intermediate from 60 to below 80, Advanced from 80.

In the embedded data, the derived levels are 6 Beginner, 8 Intermediate, and 6 Advanced. Interest counts are 6 Programming, 5 Database, 4 Web Development, and 5 Data Science. These counts describe only the 20 coded examples.



Figure 2. Interest counts in the embedded example data.

Data Preparation and Model Training

1. The function get_level assigns a label from each row's marks using the 60 and 80 thresholds.
2. LabelEncoder converts the interest and learning-style words to integer codes.
3. The feature table uses marks, attendance, study hours, interest code, and style code. The target is level.
4. train_test_split keeps 25% of the rows for testing and uses random_state=42 so the split is repeatable.
5. DecisionTreeClassifier is created with random_state=42 and max_depth=4, then fitted to the training rows.

Learning Level Rule

```
if marks < 60: Beginner elif
marks < 80: Intermediate
else: Advanced
```

8 Project Internal Structure

All project logic is kept in one Python file. There is no separate backend, frontend, or database. The internal organization follows the order of work: prepare records, create labels, preprocess values, train and evaluate the model, draw charts, define topic recommendations, collect user input, and print the predicted result.

Program block	Responsibility	Main result
Dataset setup	Creates the pandas DataFrame from fixed lists.	20 example records
Level function	Maps marks to one of three levels.	Target column
Preprocessing	Encodes interest and style; selects model columns.	Feature matrix and target
Program block	**Responsibility**	**Main result**
Training and metrics	Splits rows, fits the tree, predicts test rows.	Accuracy, report, confusion matrix
Plots	Shows evaluation and descriptive charts.	Six plot windows
Recommendation	Selects topics and a format from fixed rules.	Printed study suggestions

Console input	Checks numeric ranges and menu choices.	Learner profile and predicted level

Data Flow

1. Embedded records are placed in a DataFrame.
2. The level label is calculated from marks and category columns are encoded.
3. The tree is trained and evaluated using the prepared data.
4. The learner enters three numeric values and selects an interest and style.
5. The entered profile is converted to the model's feature columns and passed to predict.
6. The predicted level and selected interest determine topics; learning style determines the suggested format.

9 Implementation

User Input and Validation

The program asks for marks, attendance, and study hours. It checks that marks and attendance are between 0 and 100 and that study hours are not negative. It then displays numbered menus for the four interests and three learning styles. A menu choice outside the listed range prints an error message and skips the recommendation.

The numeric values are converted with float() and the menu selections with int(). The program does not catch conversion errors, so typing a word where a number is expected stops the run with a Python ValueError. Study hours have no upper bound in the input check, although the example data uses only 1 to 6 hours.

Prediction and Topic Suggestions

After valid menu selections, the two category values are encoded with the same LabelEncoder objects used for training. A one-row DataFrame is created with the model's expected five columns. The model predicts a learning level, which is printed with the entered profile.

The recommend function uses a nested dictionary. It contains four topics for each combination of four interests and three learning levels, for 48 topic entries in total. It then prints one general format: video tutorials for Video, PDF or study notes for Notes, and exercises or projects for Practical. The topic list is fixed in the code and does not include links to actual learning resources.

Example Output

A successful run prints the dataset and evaluation metrics first. It then asks for the learner's details. After valid choices, the terminal shows marks, attendance, study hours, interest, learning style, predicted learning level, four suggested topics, and a matching format. The program also opens six chart windows; close each plot window to continue through the remaining charts.

10 Testing and Results

The code calculates accuracy, a classification report, and a confusion matrix for the five held-out rows. It does not save a result file. This report describes those outputs without claiming an accuracy score, because a metric from only five test examples would be highly sensitive to individual records.

Check	Expected behavior from the code
Marks 0 to 100 and attendance 0 to 100	Passes the range check and proceeds to the menus.
Negative study hours	Prints an invalid-input message and skips the prediction.
Interest choice outside 1 to 4	Prints an invalid interest choice.
Learning style outside 1 to 3	Prints an invalid learning-style choice.
Text entered in a numeric prompt	Raises a conversion error because input conversion is not handled.
Valid profile and menu selections	Predicts a level and prints topic and format suggestions.

Charts Created by the Program

- Confusion matrix heatmap comparing actual and predicted labels for the held-out data.
- Marks histogram with a density curve.
- Study hours versus marks scatter plot, colored by the marks-derived level.
- Interest count chart and learning-level count chart.
- Correlation heatmap for numeric inputs, including integer-coded categories.

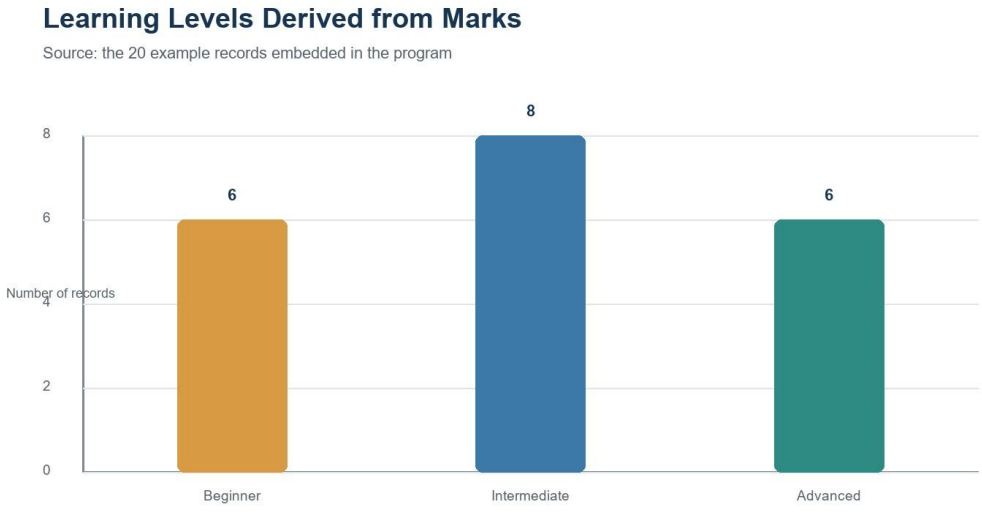


Figure 3. Level counts after applying the program's marks thresholds.

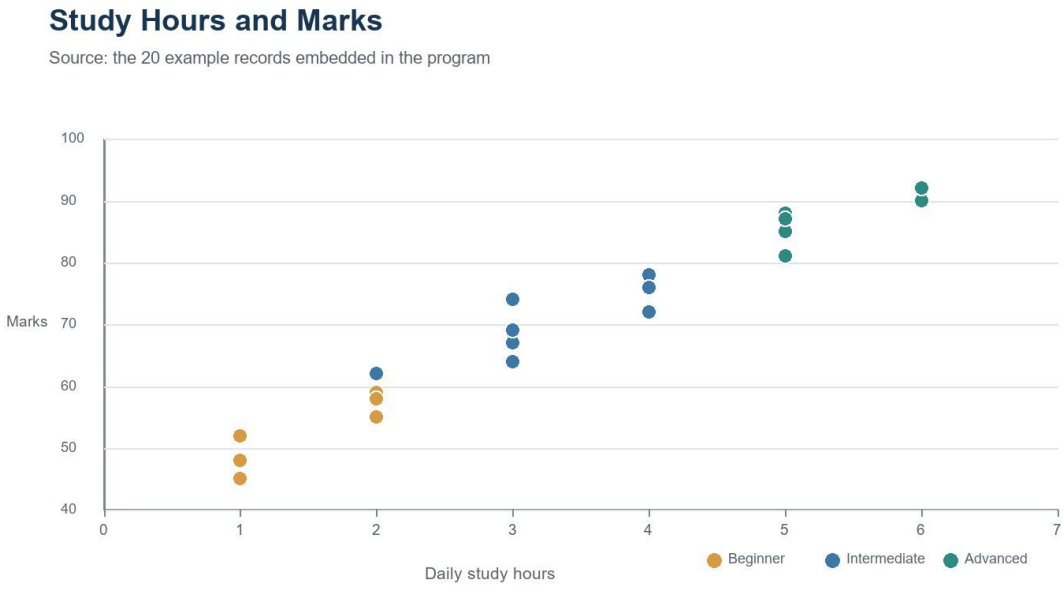


Figure 4. Example marks and study hours; color shows the level derived from marks.

How to Read the Results

Accuracy is the fraction of the held-out rows classified correctly. The classification report shows precision, recall, and F1-score by class. The confusion matrix shows counts for each actual and predicted level. With five test rows,

these values are descriptive only and should not be treated as evidence that the model will work well for a larger or different group.

11 Limitations and Risks

- Target leakage: the level is created from marks, and marks is also an input feature. The model can reproduce the written threshold rule, so its accuracy does not show independent learning assessment.
- Very small dataset: 20 examples produce only 15 training rows and 5 test rows. One row can noticeably change the evaluation scores.
- No stratified split: the small test set may not contain a representative number of each level.
- Category codes are numeric labels: LabelEncoder assigns integers to categories. Those numbers do not mean that one interest or style is greater than another.
- Correlation heatmap caution: correlations involving encoded categories can look meaningful even though the category codes are nominal labels.
- No out-of-range study-hour limit: a value much higher than the training examples is accepted, even though the model has not seen such values.
- No text-input recovery: letters or blank input at numeric prompts are not handled with a friendly retry message.
- Static recommendations: topics and formats are predefined; the program does not check their quality, availability, or suitability and provides no URLs.
- No persistence or consent flow: profiles and feedback are not stored, and the script does not request consent for collecting data.

12 Future Scope

- Replace the small hard-coded examples with a larger, permission-based dataset and document how it was collected.
- Define learning outcomes independently of marks to prevent the model from simply repeating the target rule.
- Use a stratified split or cross-validation and report per-class results with the sample size beside each metric.
- Use a categorical preprocessing method such as one-hot encoding and fit all preprocessing steps only on the training data.
- Add safe input parsing, clear retry prompts, and practical limits for attendance, marks, and study hours.
- Add real learning resources with subject, level, format, and link metadata, then check that links remain available.
- Collect optional topic ratings and later learning outcomes with informed consent to evaluate whether recommendations help.
- Build a simple graphical or web interface and add opt-in profile storage with appropriate privacy controls.

13 Conclusion

The project demonstrates the main steps of a small classification and recommendation program. It prepares example learner records, derives three levels, trains a limited-depth decision tree, accepts a profile in the console, and prints topic suggestions based on interest and predicted level. Its charts and classification metrics help explain a machine learning workflow to a beginner.

The project meets its practical demonstration goals, but the current dataset and target rule limit what can be concluded from the model. A larger, consented dataset, an independently defined learning outcome, safer input handling, and verified resource links would be needed before the system could support real learner decisions.

14 References

[1] Submitted source code: 1 (1).py.

[2] Brainware University, Bachelor of Computer Application syllabus, 2024. Machine Learning course listing, codeBCA57205. https://www.brainwareuniversity.ac.in/syllabus/2024/BCA%202024_dacbc6955e.pdf

[3] Scikit-learn documentation. Decision trees, model selection, preprocessing, and classification metrics.https://scikit-learn.org/stable/

[4] pandas documentation. DataFrame and data analysis tools. https://pandas.pydata.org/docs/

[5] Seaborn documentation. Statistical data visualization. https://seaborn.pydata.org/

[6] Matplotlib documentation. Python plotting library. https://matplotlib.org/stable/

How to Run the Program

Install Python and the required libraries pandas, scikit-learn, matplotlib, and seaborn. Save the submitted script in a folder and open a terminal in that folder. Run it with Python, keeping quotation marks around the filename because it contains spaces and parentheses:

```
python "1 (1).py"
```

The program is interactive. Enter numeric values at the prompts and select a listed menu number. Each chart opens in a separate window; close the current chart window to view the next one.